ESE 124 Final Exam - Fall 2025

Instructor: Jenny Chen **Duration: 2.5 Hours**

- First and Last Name: _____
- ID: ___
- Lab Section: ____

Instructions:

Choose 3 out of the 4 questions to answer. If you answer all 4, the best 3 will be graded.

Bonus: +15 points if all 4 are answered perfectly.

Write clearly and comment your code where necessary.

General Notes:

You may use: stdio.h, stdlib.h, string.h, ctype.h.

Free all dynamically allocated memory.

Handle file errors and NULL pointers.

Question 1: File Token Counter (33.33 points)

Write a program that reads a text file and counts words and numbers using a finite state machine.

Function to implement:

```
typedef struct {
    int wordCount;
    int numberCount;
} TokenStats;

int scanFileTokens(const char* filename, TokenStats* stats);
```

Returns 1 on success, 0 on failure.

Requirements: Use enum for states (START, WORD, NUMBER). Words are sequences of letters. Numbers are sequences of digits. Use `isalpha()` to detect letters and `isdigit()` to detect digits.

Test Case:

input.txt:

```
hello ESE_124
abc + 456 - def
```

Output:

```
Words: 4
Numbers: 2
```

Question 2: Student Grade Report. (33.33 points)

Read student data from a file and generate a report showing course averages and struggling students.

Structs:

```
typedef struct {
    char name[50];
    float midterm;
    float final;
} Student;

typedef struct {
    char courseCode[20];
    int numStudents;
    Student* students;
} Course;
```

Functions to implement:

```
Course* loadCourses(const char* filename, int* numCourses);
void sortCourses(Course* courses, int numCourses);
void printReport(Course* courses, int numCourses);
void cleanup(Course* courses, int numCourses);
```

loadCourses reads the file and allocates memory. Calculate overall grade as: 0.4 $*$ *midterm* + *0.6* $*$ final.

sortCourses sorts by average overall grade (highest first) using bubble sort.

printReport displays each course with its average, then lists students with overall < 65.

cleanup frees all memory.

Test Case:

grades.txt:

```
2
CSE214 3
Alice 85.0 90.0
Bob 50.0 60.0
Charlie 75.0 80.0
MAT126 2
David 70.0 75.0
Emma 55.0 50.0
```

File format: First line is number of courses. For each course: course code, number of students, then one line per student with name, midterm, final.

Output:

```
Course: CSE214
Average: 76.33
Struggling Students:
  Bob: 56.00

Course: MAT126
Average: 65.00
Struggling Students:
  Emma: 52.00
```

Question 3: Bracket Validator with a Stack ADT. (33.33 points)

Check if brackets in an expression are balanced using a **stack ADT.**

Given stack.h:

```
typedef struct {
    char data[100];
    int top;
```

```
} Stack;

void initStack(Stack *s);
int push(Stack *s, char c);
int pop(Stack *s, char *c);
int isEmpty(Stack *s);
```

Function to implement:

```
int isBalanced(const char* expr);
```

Returns 1 if balanced, 0 if not. Valid pairs: (), [], {}.

Test Cases:

```
isBalanced("(a + b)")        → 1
isBalanced("{[()]}")         → 1
isBalanced("[(])")           → 0
isBalanced("((a)")           → 0
```

Implement the stack functions and isBalanced. Ignore non-bracket characters.

Question 4: Implement a Queue-Based Task Processing System using a Queue ADT. (33.33 points)

The program should demonstrate at least three different task types (e.g., print, add digits, reverse string) **where each task stores its own handler function pointer**. Your solution should be split into three files: queue.c queue.h and main.c

queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

typedef struct{
    char data[50];
    void (*handler)(char*);
} Task;

typedef struct {
    Task tasks[50];
```

```
    int front;
    int rear;
    int size;
}Queue;

void initQueue(Queue *q);
int isFull(Queue *q);
int isEmpty(Queue *q);
void enqueue(Queue *q, Task t);
int dequeue(Queue *q, Task *t);

#endif
```

main.c

```
int main(){
    Queue q;
    initQueue(&q);

    Task t1 = {"Hello, ESE124!", handPrint};
    Task t2 = {"12345", handAdd};
    Task t3 = {"ABCDE", handReverse};

    enqueue(&q, t1);
    enqueue(&q, t2);
    enqueue(&q, t3);

    processTasks(&q);

    return 0;
}
```

output:

Print Task: Hello, ESE124!
Add Task: Sum = 15
Reverse Task: EDCBA